



Informe de resultados

Transformación de granularidad mediante un pipeline de análisis exploratorio

Tercer cuatrimestre
Aprendizaje no Supervisado
Dr. Ernesto García Amaro

Jessica Melani Romero Lora

24 de mayo del 2026

Tabla de contenido

1. Introducción.....	3
2.1 Determinación de granularidad.....	5
2.2 Transformación del pipeline.....	5
2.3 Tratamiento de valores faltantes.....	6
3.1 Análisis Univariado.....	7
3.1 Estructura de correlación cruzada.....	7
5. Discusión de Resultados.....	8
6. Conclusiones.....	9
Bibliografía.....	9

1. Introducción

Frecuentemente, los pipelines analíticos se enfrentan a bases de datos de comportamiento con un nivel de granularidad superior al de la unidad de análisis objetivo. Este fue el caso observado para el consumo de entretenimiento, en donde cada fila no logra reunir el perfil completo del estudiante, al operar un dataset en formato largo sin transformación previa no lograría modelar la computación de las comparaciones computacionales, obtener las estadísticas multivariadas o el despliegue de modelos de Machine Learning.

El objetivo del presente reporte es lograr construir un pipeline que en primer lugar, reduzca 600 filas a 150 para los registros de estudiantes y posteriormente, derivar los resultados del análisis exploratorio a una base de datos de calidad. En la sección 2 del documento se describen los problemas de calidad que presenta. Posteriormente en la sección 3 se detalla la metodología de transformación, mientras que la sección 4 se presenta el análisis exploratorio de datos, incluyendo estadística univariada y la estructura detrás de la correlación. Finalmente, la estructura 5 discute las implicaciones analíticas de los hallazgos.

Finalmente, en el presente documento se presentan los resultados de una transformación de un pipeline de datos enfocado en consumo de entretenimiento diverso en estudiantes, el dataset originalmente mantenía una estructura de formato largo (600 filas, 3 columnas), derivando una estructura de formato ancho.

Los datos presentaron diversos problemas de calidad como un separador de coma europeo incompatible con el procesamiento tradicional requerido por los datos. Posteriormente, se identificaron problemas en la columna de libros o *books*, por lo que se aplicaron diversas técnicas como **pivot_table**, logrando una reducción de 150 filas una proporción equivalente a uno por estudiante y cuatro variables numéricas representando las horas dedicadas al consumo de contenido como videojuegos, televisión, programas de televisión, películas y libros.

Durante la etapa de análisis exploratorio de datos se identificaron datos heterogéneos entre categorías: videojuegos acumulo el promedio mas alto con 5.84hrs por semana, mientras que libros registro la más baja con un promedio de 3.10hrs por semana. Por otra parte, se observó una desviación estándar aproximada de 2.7 veces superior a la categoría de videojuegos, evidenciando la segmentación notoria en el dataset.

Posteriormente, se aplicó una correlación de Pearson para confirmar la segmentación de clusters entre estudiantes consumidores de videojuegos, películas y libros, esta última categoría exhibió un grado de correlación de $r > 0.78$, lo cual contrasta con una relación negativa o cercana a cero con los programas de televisión. Estos resultados sugieren que la base de datos fue generada de manera sintética, esto tiene implicaciones importantes porque, al ser un dataset sintético existen riesgos evidentes para la generalización de resultados. Estructura original de la tabla:

Variable	Tipo	Descripción	Rango
nombre	string	Vectorización NumPy (.flatten()) + lectura directa en gris	-
entretenimiento	categorica	Proceso dedicado por clase; eliminación de contención en escritura	-
horas_por_semana	float64	Medición de tiempo con time.perf_counter() y speedup real	0.5 - 7.9

Durante la inspección de calidad se identificaron problemas de calidad severos, en primer lugar la columna de **horas_por_semana** incluía comas de estilo europeo como separador decimal designado (p. ej, "5,1" en lugar de 5.1), esto ocasiona que la librería pandas no pueda leer de manera adecuada el objeto de tipo dtype, lo cual produce NaN's durante la conversión numérica directa, incluyendo la reconversión.

En segundo lugar, se encontraron 5 valores faltantes de la columna libros, estos valores se concentraban en los nombre de estudiantes que tenían un orden alfabético en la base de datos. El patrón de datos faltantes sugiere una omisión sistemática evidente de la recolección de datos más que una caída aleatoria, la cual es una limitación para el desarrollo del proyecto.

2. Metodología

2.1 Determinación de granularidad

La estructura objetivo es de 150 filas determinada por el número de estudiantes, mientras que cada estudiante aparece 4 veces en el formato largo es decir, una vez por cada categoría de entretenimiento, esto nos sugiere que la transformación necesaria es modificar la estructura a una base de datos más ancha, más no una agregación de los datos o la sumatoria de los mismos. Esta distinción es consecencial, en el sentido de, si se aplicara una función **groupby** se destruirían los valores numéricos, mientras que la función **pivot** preservará los valores como columnas de categorías específicas.

```
entertainment_pipeline.py •
entertainment_pipeline.py > ...
1 import pandas as pd
2 import numpy as np
3 # — 1. Importar datos
4 df = pd.read_csv("entertainment.csv")
5 print(f"[1] Shape original (long): {df.shape}")
6 print(df.head(4), "\n")
7
```

2.2 Transformación del pipeline

En el código implementado se utilizó la librería pandas, cargando los datos en formato CSV con el objetivo de transformarlos en un dataframe que mantenía la forma (600, 3). Posteriormente se realizó un reemplazo de datos string en la columna de **horas_por_semana** utilizando la función **str.replace(',', '.')** seguido por la función de **pd.to_numeric()** con **errors='coerce'** para transformarlo a float64.

```
8 # — 2. Limpiar separador decimal
9 df["hours_per_week"] = (
10 | df["hours_per_week"].astype(str).str.replace(",", ".").str.strip()
11 | )
12 df["hours_per_week"] = pd.to_numeric(df["hours_per_week"], errors="coerce")
13 print(f"[2] Nulos detectados tras cast:\n{df.isna().sum()}\n")
```

Este paso confirma la unidad analítica al inspeccionar la cardinalidad del nombre del estudiante a 150 valores únicos, las columnas categóricas son cuatro únicamente, por lo que, mediante la función **pd.pivot_table()** con el índice **index='name'**, **columns='entretenimiento'**, **values='horas_por_semana'** y la **aggfunc='mean'**.

```
# — 3. Determinar granularidad: 1 fila por estudiante
print(f"[3] Estudiantes únicos: {df['name'].nunique()}")
print(f"    Categorías únicas: {df['entertainment'].unique()}\n")

# — 4. Pivot long → wide
df_wide = df.pivot_table(
    index="name",
    columns="entertainment",
    values="hours_per_week",
    aggfunc="mean",
).reset_index()
df_wide.columns.name = None
df_wide = df_wide[["name", "video_games", "tv_shows", "movies", "books"]]
print(f"[4] Shape tras pivot (wide): {df_wide.shape}")
```

La función de agregación promedio asegura el comportamiento determinista en la presencia de duplicados, sin embargo, no se detectó ninguno en el dataset. Finalmente, el último paso le da forma y exporta el resultado a formato csv bajo el nombre `entertainment_wide.csv`.

2.3 Tratamiento de valores faltantes

Se descubrieron 5 valores faltantes en la columna de libros (representando únicamente el 3.3% de la variable). Dado que `horas_por_semana` es una variable de escala variable y la distribución de la columna es simétrica con un promedio de 3.10 y una mediana de 3.20, el promedio de la imputación es seleccionado como estrategia transparente para mantener una correcta manipulación de los datos. El valor imputado es de 3.10hrs por semana, lo cual está documentado en el logaritmo de transformación. Este patrón sugiere un análisis de sensibilidad o múltiples imputaciones.

```
30 # — 5. Imputar nulos con media de columna —————
31 print(f"\n[5] Nulos por columna antes de imputación:\n{df_wide.isna().sum()}")
32 ✓ for col in ["video_games", "tv_shows", "movies", "books"]:
33     col_mean = round(df_wide[col].mean(), 2)
34     df_wide[col] = df_wide[col].fillna(col_mean)
35 print(f"\n    Nulos tras imputación:\n{df_wide.isna().sum()}\n")
36
37 # — 6. Verificar y guardar —————
38 print(f"[6] Shape final: {df_wide.shape} ← target: (150, 5)")
39 print(df_wide.head(5), "\n")
40 print("[6] Estadísticas descriptivas:")
41 print(df_wide.describe().round(3))
42
43 df_wide.to_csv("entertainment_wide.csv", index=False)
44 print("\n✓ Guardado: entertainment_wide.csv")
45
```

3. Análisis exploratorio de los datos

3.1 Análisis Univariado

La presente tabla presenta un análisis descriptivo estadístico de las cuatro categorías en la base de datos posteriormente transformada. En donde, la categoría de videojuegos representa el promedio de consumo semanal más alto de 5.84hrs y una desviación estándar de ($\sigma = 0.83$). Los programas de televisión mantienen una posición intermedia de 4.59hrs en promedio a la semana, con una desviación estándar de ($\sigma = 0.65$).

Las películas exhiben la desigualdad de consumo más grande entre los estudiantes con un rango intercuartílico de 1.60 a 5.10 hrs. Esto sugiere una distribución bimodal o multimodal consistente con la presencia de distintos perfiles de consumo entre los perfiles de estudiantes en la muestra.

Statistic	video_games	tv_shows	movies	books
n	150	150	150	150
Mean	5.843	4.586	3.758	3.097
Std Dev	0.828	0.646	1.765	1.835
Min	4.300	3.000	1.000	0.500
Q1 (25%)	5.100	4.200	1.600	1.000
Median	5.800	4.500	4.350	3.200
Q3 (75%)	6.400	4.900	5.100	4.500
Max	7.900	6.600	6.900	6.200

Tabla 1. Estadísticas descriptivas v ($n = 150$).

3.1 Estructura de correlación cruzada

La Tabla 3 muestra la matriz de correlación de Pearson entre las cuatro variables de entretenimiento. Tres hallazgos destacan, primero, los videojuegos, las películas y los libros presentan una fuerte correlación positiva entre sí (r entre 0,785 y 0,929), formando un grupo cohesionado de alto consumo. Segundo, los programas de televisión muestran una débil correlación negativa con las tres variables (r entre -0,116 y -0,424), lo que sugiere que funcionan como sustitutos en lugar de complementos de las otras actividades. Tercero, la correlación entre películas y libros ($r = 0,929$) es notablemente alta —cercana a la colinealidad—, lo

que debería considerarse en cualquier modelo de regresión que incluya ambos como predictores.

	video_games	tv_shows	movies	books
video_games	1.000	-0.116	0.872	0.785
tv_shows	-0.116	1.000	-0.424	-0.343
movies	0.872	-0.424	1.000	0.929
books	0.785	-0.343	0.929	1.000

Table 2. Matriz de correlación de Pearson

5. Discusión de Resultados

Los resultados exploratorios apuntan consistentemente a un conjunto de datos sintéticos generado a partir de al menos dos o tres grupos poblacionales distintos con perfiles de consumo diferenciados: un grupo de bajo consumo concentrado en los cuartiles inferiores de películas y libros (por debajo del cuartil 1), un grupo de consumo medio cercano a la mediana y un grupo de alto consumo cercano al cuartil 3 y superior.

Esta estructura visible en las grandes desviaciones estándar y los amplios rangos intercuartil de películas y libros, la brecha mediana bimodal en películas (media 3,76, mediana 4,35) y la casi colinealidad entre películas y libros es característica de la generación de datos sintéticos estratificados, en lugar de datos de comportamiento naturalistas.

Esta observación tiene una implicación directa para la validez externa: cualquier modelo entrenado con este conjunto de datos podría capturar los límites entre estratos sintéticos en lugar de relaciones de comportamiento genuinas. La fuerte correlación entre películas y libros ($r = 0,929$) justifica una atención especial en la selección de características para cualquier tarea posterior de clasificación o regresión, ya que incluir ambas variables en un modelo lineal introduciría una multicolinealidad severa.

El proceso de transformación documentado en la Sección 3 es totalmente reproducible a partir del archivo CSV original. La estructura de seis pasos (cargar, limpiar, inspeccionar, pivotar, imputar y verificar) constituye una plantilla reutilizable aplicable a cualquier conjunto de datos de comportamiento en formato largo. Cabe mencionar que el campo "nombre" se utiliza como identificador único del estudiante; en un entorno real, esta clave sería poco fiable, susceptible a errores ortográficos y duplicados, por lo que debería sustituirse por un identificador numérico.

6. Conclusiones

Este informe presentó un proceso reproducible de seis pasos para transformar un conjunto de datos de entretenimiento de formato largo de 600 filas en una estructura de formato ancho de 150 filas a nivel de estudiante, adecuada para el análisis multivariante. Se identificaron, documentaron y resolvieron dos problemas de calidad de datos: el formato decimal europeo y la presencia de valores faltantes no aleatorios en la categoría de libros, antes de la transformación.

El análisis exploratorio del conjunto de datos transformado revela que los videojuegos predominan en el consumo semanal promedio, pero presentan la distribución más uniforme, mientras que las películas y los libros muestran una alta dispersión, consistente con la segmentación latente. La estructura de correlación de Pearson identifica un fuerte grupo positivo entre videojuegos, películas y libros, en contraposición a una relación casi nula o negativa con los programas de televisión.

Estos hallazgos sugieren que los métodos de segmentación no supervisada —agrupamiento k-means o análisis de componentes principales— serían un siguiente paso productivo, ya que podrían recuperar los grupos de población sintéticos subyacentes y permitir un modelado posterior más interpretable.

Bibliografía

McKinney, W. (2010). Data structures for statistical computing in Python. Proceedings of the 9th Python in Science Conference, 51–56.

pandas development team. (2024). pandas: Powerful Python data analysis toolkit (v2.x). <https://pandas.pydata.org>

Wickham, H. (2014). Tidy data. Journal of Statistical Software, 59(10), 1–23. <https://doi.org/10.18637/jss.v059.i10>